

Project Details Table

Sr. No.	Details	Description
1	Project Title	Register VM-Based Reactivity Engine: A Bytecode-Driven Runtime for Fine-Grained Frontend UI Updates (RORE)
2	Problem Statement	Modern frontend frameworks rely on Virtual DOM reconciliation, proxy interception, or closure-based dependency graphs to achieve reactivity. Each approach carries a structural cost: frequent allocation of short-lived signal and closure objects increases garbage-collection pressure, structural diffing adds CPU overhead on every update, and polymorphic execution paths can trigger JIT de-optimization in engines like V8 and SpiderMonkey. No widely adopted web runtime currently optimizes reactive updates at the instruction level rather than the JavaScript object level. This project addresses that gap by treating UI reactivity as a compilation target instead of a runtime abstraction.
3	Project Objective(s)	• Define a compact instruction set architecture (ISA) purpose-built for UI operations, register-based variable matching, and DOM patching. • Build an ahead-of-time (AOT) compiler pipeline that compiles declarative templates directly into a dense, linear bytecode stream. • Implement a low-allocation register virtual machine backed by flat ArrayBuffers/TypedArrays, avoiding runtime object allocation. • Achieve fine-grained DOM synchronization by mapping state mutations directly to virtual registers, without Virtual DOM traversal. • Provide runtime instrumentation to profile instruction dispatch overhead, memory footprint, and execution throughput.
4	Project Domain	• Compiler Design & Engineering • Virtual Machines & Runtime Systems • Systems-Oriented Web Development • Performance Engineering & Benchmarking
5	Project Type	Experimental Frontend Runtime System / Systems-Oriented Research Project
6	Target Users	• Frontend runtime engineers evaluating alternatives to

Sr. No.	Details	Description
		reactive-graph and Virtual DOM architectures. • Systems researchers studying virtual-machine execution within constrained, single-threaded browser sandboxes. • Computer science students and educators focused on compiler backend design, intermediate representations, and register allocation.
7	User Roles	• Developer / Researcher – designs and writes declarative, reactive UI templates. • Developer / Researcher – analyzes compiled bytecode structures and evaluates register-allocation efficiency. • Developer / Researcher – configures and runs compiler optimization passes. • Developer / Researcher – monitors VM state, register files, and execution profiles during runtime debugging.
8	Major Modules	• Template Parser & AST Builder – parses declarative UI templates into a strict abstract syntax tree. • Static Analyzer & Reactivity Mapper – separates static structure from dynamic state bindings before compilation. • Bytecode Emitter & Register Allocator – generates linear instruction streams and assigns state to virtual registers. • Register VM Core (Interpreter) – a tight fetch-decode-execute loop operating on typed-array memory spaces. • Bitmask Dependency Matrix – a low-allocation lookup table routing mutated register IDs to instruction targets. • Granular Element Renderer – translates VM patching opcodes into explicit, direct DOM mutations. • Performance Profiler – captures bytecode density, memory consumption, and instruction execution-time metrics.
9	References	• Nystrom, R. (2021). Crafting Interpreters – architectural grounding for register-allocated VM execution loops. • Cooper, K. D., & Torczon, L. (2011). Engineering a Compiler – data-flow analysis and register allocation strategies. • Ierusalimschy, R., de Figueiredo, L. H., & Celes, W. (2005). The Implementation of Lua 5.0 – register-based interpreter design. • SolidJS and Svelte core architectures – prior art on

Sr. No.	Details	Description
		compile-time dependency tracking and fine-grained DOM manipulation.